

Module 1 -- Linux Foundations to Kernel Internals

Complete Study Reference | DevOps Mastery Roadmap, Phase 1

FULL TOPIC MINDMAP (17 Sections -- 11 Original + 6 Gap-Filled)

MODULE 1: LINUX FOUNDATIONS -> KERNEL INTERNALS

```
|
+-- 1. Linux Landscape
|   +-- History & Philosophy (Unix lineage, GNU/Linux, open source)
|   +-- Distro Families (Debian/apt, RHEL/dnf, Arch/pacman, SUSE/zypper)
|   +-- Choosing a distro: server vs desktop
|
+-- 2. System Architecture Layers
|   +-- Hardware -> Kernel Space [OS] -> Syscall Interface [OS] -> Userspace ->
Apps
|
+-- 3. The Boot Process [OS: bootstrapping]
|   +-- Firmware (BIOS/UEFI+Secure Boot) -> GRUB2 -> Kernel init/initramfs ->
systemd
|
+-- 4. Filesystem Hierarchy Standard (FHS)
|   +-- /bin /etc /home /var /usr /tmp /opt /proc [OS] /sys [OS] /dev /lib
|
+-- 5. Disk & Filesystem Management [GAP-FILLED]
|   +-- lsblk, mount, /etc/fstab, df vs du
|
+-- 6. Navigation & File Operations
|   +-- Navigation/Search/File ops, Hard vs Soft Links [OS: inodes], vim,
less/more
|
+-- 7. I/O Redirection & Pipes [GAP-FILLED]
|   +-- stdin/stdout/stderr, >, >>, 2>&1, pipes |, tee, /dev/null
|
+-- 8. Permissions & Ownership [GAP-FILLED]
|   +-- rwx model [SEC], chmod, chown, umask, SUID/SGID/sticky bit
|
+-- 9. Users & Groups [GAP-FILLED]
|   +-- /etc/passwd, /etc/shadow, /etc/group, useradd/usermod, sudo vs su
|
+-- 10. Package Management
```

```
|   +-- apt/dpkg (Debian), dnf/yum/rpm (RHEL), GPG signing [SEC], compiling from
source
|
+-- 11. systemd Deep Dive
|   +-- Unit types, systemctl, journalctl, custom .service files, timers vs cron
|
+-- 12. Kernel Interfaces to Userspace [OS]
|   +-- /proc, /proc/<pid>/, /proc/sys, /sys, sysctl [PERF]
|
+-- 13. System Call Interface [OS: syscall interface, mode switching]
|   +-- fork/exec/exit/wait, open/read/write/close, ioctl, strace/ltrace
|
+-- 14. Process Management & Signals [GAP-FILLED]
|   +-- ps/top/htop, process states, kill/signals, job control, nice/renice
|
+-- 15. Kernel Modules [OS: kernel extensibility]
|   +-- lsmod, modprobe/insmod/rmmod, modinfo, monolithic+modules model
|
+-- 16. cgroups & Namespaces [GAP-FILLED]
|   +-- Namespaces (PID/NET/MNT/UTS/IPC/USER), cgroups, how containers work
|
+-- 17. Module 1 Capstone Checkpoint
    +-- Boot narration, strace task, custom .service task, /proc exploration,
cgroup exploration
```

Tags Used in This Document

- **[OS]** -- Operating Systems (processes, memory, scheduling, filesystems, concurrency)
- **[SEC]** -- Security fundamentals (access control, cryptographic trust chains)
- **[PERF]** -- Performance / systems engineering (runtime tuning)
- **[GAP-FILLED]** -- Sections added during the gap-check that were not in the original file

SECTION 1: Linux Landscape

1.1 History & Philosophy

Linux began in 1991 when Linus Torvalds released a free kernel he had written as a hobby project, inspired by Minix and the broader Unix design philosophy from the 1970s. The GNU Project, started by Richard Stallman in 1983, had already spent nearly a decade building the userspace tools (compilers, shells, utilities) an operating system needs, but was missing a working kernel. Linux filled that missing piece -- which is why purists call the whole system "GNU/Linux": the kernel is Linux, but most everyday tools (bash, grep, gcc, ls) are GNU software wrapped around it.

The Unix philosophy shapes almost every design decision in this module: write programs that do one thing well, make programs work together, and prefer text streams as the universal interface between them. The open-source model behind it means the kernel source is publicly available under the GPL license, which is a large part of why Linux ended up running everything from phones (Android) to the majority of the world's servers and supercomputers.

1.2 Distro Families

A distribution bundles the Linux kernel with a package manager, default software, and configuration conventions. The kernel is nearly identical across distros -- what differs is how software is installed and managed.

Family	Examples	Package Format	High-Level Manager	Low-Level Manager
Debian-based	Ubuntu, Debian, Mint	.deb	apt	dpkg
RHEL-based	RHEL, CentOS, Fedora, Rocky, Alma	.rpm	dnf (modern) / yum (older)	rpm
Arch-based	Arch Linux, Manjaro	plain tarballs	pacman	--
SUSE-based	openSUSE, SLES	.rpm	zypper	rpm

⚠ NOTE: CentOS as a stable general-purpose server distro effectively ended in 2021 when Red Hat shifted it into 'CentOS Stream' (a rolling preview of RHEL, not a stable downstream release). Rocky Linux and AlmaLinux were created specifically to fill the gap CentOS left -- a pre-2021 tutorial recommending CentOS for production is now outdated.

1.3 Choosing a Distro: Server vs Desktop

Servers prioritize stability, long support windows, and predictable behavior -- which is why Ubuntu Server LTS, Debian, and RHEL/Rocky dominate production. LTS releases receive years of security patches without changing core package versions underneath you. Desktops prioritize hardware support, UI polish, and up-to-date software -- Ubuntu Desktop, Fedora, and Mint are common entry points, with Fedora tracking newer kernel/package versions more aggressively. As a DevOps engineer, the overwhelming majority of work happens on Debian/Ubuntu or RHEL-family servers.

The Command

```
cat /etc/os-release
```

cat -> print file contents to standard output

/etc/os-release -> standardized file storing machine-readable distro identity: name, version, ID

Tie-back

This section is the foundation the rest of Module 1 sits on: Section 10 (Package Management) is the direct practical consequence of distro family, and boot/systemd behavior is otherwise identical across families.

Key Takeaway

The distro determines the package manager and release philosophy, not the kernel's behavior. Debian-family uses apt/.deb, RHEL-family uses dnf/.rpm, and the underlying kernel and system administration is otherwise the same.

Section 1 Summary Table

Topic	Level	Key Command	One-line Takeaway
History & Philosophy	Basic	--	Linux = kernel (Linus) + GNU tools; small composable tools philosophy
Distro Families	Basic	cat /etc/os-release	Debian->apt/.deb, RHEL->dnf/.rpm, Arch->pacman, SUSE->zypper
Server vs Desktop	Basic	--	Servers prioritize LTS stability; desktops prioritize fresh packages

SECTION 2: System Architecture Layers

2.1 The Layered Model

Every Linux system is a stack of layers, each only able to talk to the layer directly next to it: Hardware -> Kernel Space -> System Call Interface -> Userspace/Shell -> Applications. This isolation is deliberate: a bug in a text editor cannot directly corrupt a disk controller, and new hardware does not require every application to be rewritten.

Hardware is the physical substrate -- CPU, RAM, storage, network interfaces -- with no concept of 'files' or 'processes'. Kernel space is the privileged layer managing scheduling, memory ownership, filesystems, and device drivers.

[OS] -> this is the principle of privilege separation -- the CPU has hardware-enforced privilege levels (rings on x86; the kernel runs in the most privileged ring, ring 0), so even a misbehaving kernel component cannot be bypassed by user code demanding more access than allowed.

The System Call Interface is the single, narrow doorway between userspace and kernel space -- a regular program is never allowed to directly read a disk sector; it must ask the kernel via a syscall like read() or write(). Userspace/Shell is where bash, glibc, and utilities live. Applications sit at the top, relying on everything below them.

⚠ NOTE: A common beginner misconception is thinking 'userspace' means 'less important' than kernel space. It is a hierarchy of privilege, not importance -- 99% of software, including production servers and databases, runs entirely in userspace.

2.2 Why This Separation Exists

[SEC] -> this layering is a foundational security principle -- access control by privilege level. Forcing every hardware interaction through the kernel via the syscall interface lets the kernel enforce rules such as 'process A cannot read process B's memory.'

Comparison: Kernel Space vs Userspace

Aspect	Kernel Space	Userspace
Privilege level	Highest (ring 0 on x86)	Restricted (ring 3 on x86)
Direct hardware access	Yes	No -- must go through syscalls
Memory access	All physical memory	Confined to its own virtual address space
Failure impact	Can crash/hang entire system	Process crashes in isolation
Examples	Scheduler, memory manager, drivers	bash, nginx, python3, applications

Tie-back

This section is the conceptual skeleton for the whole module: the Boot Process is the kernel taking control of hardware and handing off to userspace; /proc and /sys are the kernel exposing kernel-space information to userspace; and the System Call Interface is the formal mechanism for the boundary introduced here.

Key Takeaway

The syscall interface is not a convenience, it is a security and stability boundary enforced by the CPU hardware itself, not just software convention.

? Doubt Clarified (re: System Architecture Layers -- Privilege Separation & Ring 0): Can you explain more about the principle of privilege separation and what it actually means for the kernel to sit on ring 0 on x86?

Privilege separation means not all code should be trusted equally, and the hardware itself enforces that distrust rather than relying on software convention alone. x86 CPUs define four protection rings (0-3) baked into the silicon; Linux uses only ring 0 (kernel mode, full hardware access including privileged instructions) and ring 3 (user mode, barred from privileged instructions at the hardware level -- attempting one raises a general protection fault and the CPU kills the process before harm occurs). Rings 1 and 2 exist in the x86 spec but sit unused in Linux's simpler two-tier model. This is exactly why the system call interface must exist as a formal mechanism: a ring 3 program cannot jump into ring 0 code on its own. The CPU provides a special instruction (historically int 0x80, now syscall/sysenter on x86-64) that safely transitions from ring 3 to ring 0, but only by jumping to an exact, kernel-defined entry point chosen in advance -- never an arbitrary address the calling program

picks. The ring model itself predates Linux by decades, tracing back to the Multics OS design and implemented in x86 starting with the 80286 in 1982.

Section 2 Summary Table

Layer	Role	Privilege	Tag
Hardware	Physical CPU/RAM/disk/NIC	--	--
Kernel Space	Process/memory mgmt, drivers, filesystems	Highest (ring 0)	OS
System Call Interface	Controlled doorway user<->kernel	Boundary	OS
Userspace/Shell	Interactive interface, libc, utilities	Restricted (ring 3)	--
Applications	User-run programs	Restricted (ring 3)	--

SECTION 3: The Boot Process [OS: bootstrapping]

3.1 Firmware Stage

The first code that runs on power-on is firmware, not Linux. Its job is to initialize enough hardware (CPU, RAM, boot device) to find and hand control to a bootloader.

BIOS (legacy) runs in 16-bit real mode, performs a Power-On Self-Test, then looks at the Master Boot Record (first 512 bytes of the boot disk) for bootloader code. UEFI (modern) runs in 32/64-bit mode, supports much larger disks (BIOS/MBR caps around 2TB), and reads boot data from a dedicated EFI System Partition -- a real filesystem rather than raw bytes at a fixed offset. UEFI also introduces Secure Boot, a cryptographic verification chain that only executes a bootloader signed by a trusted key, blocking bootkits that install malicious code below OS-level security.

Aspect	BIOS (legacy)	UEFI (modern)
Mode	16-bit real mode	32/64-bit
Boot data location	MBR (first 512 bytes)	EFI System Partition (real filesystem)
Max disk size	~2TB (MBR limit)	Effectively unlimited (GPT)
Security	None built-in	Secure Boot (signature checks)
Boot speed	Slower	Generally faster

[SEC] -> Secure Boot is a cryptographic trust chain: each stage (firmware -> bootloader -> kernel) verifies the next stage's signature before executing it, so a compromise cannot silently insert itself earlier than the verification starts.

3.2 Bootloader Stage

GRUB2 is the bootloader on the overwhelming majority of Linux systems today. It presents a menu, loads the kernel image and initramfs into memory, and passes boot parameters before jumping into the kernel.

```
/etc/default/grub    -> human-edited SOURCE settings
/boot/grub/grub.cfg  -> auto-GENERATED file GRUB actually reads at boot
```

⚠ NOTE: A very common mistake is editing /boot/grub/grub.cfg directly -- it is auto-generated and manual edits get silently overwritten. Edit /etc/default/grub, then run update-grub (Debian/Ubuntu) or grub2-mkconfig -o /boot/grub2/grub.cfg (RHEL family) to regenerate it.

3.3 Kernel Initialization

The kernel image is decompressed into memory and the kernel begins detecting and initializing hardware. The chicken-and-egg problem this stage solves: the kernel needs driver modules to reach the real root filesystem, but those modules usually live on that root filesystem. initramfs (the modern replacement for initrd) is a small, temporary filesystem loaded into RAM alongside the kernel, containing just enough drivers/tools to detect, decrypt, or assemble whatever is needed to mount the real root filesystem. Once mounted, the kernel switches over (pivot_root/switch_root) and discards the temporary initramfs.

[OS] -> this is a direct example of bootstrapping with a minimal, temporary environment because the full environment is not yet reachable -- a pattern that also appears in container startup and network boot (PXE) scenarios.

3.4 Init System Takeover

Once the real root is mounted, the kernel launches PID 1 -- the init system -- and steps back into a supporting role. systemd is the modern default init system on essentially every major distro, replacing the older SysVinit, which started services sequentially via numbered scripts in /etc/init.d/. systemd starts services in parallel wherever dependencies allow, tracks state with cgroups, and uses a declarative unit-file model instead of imperative scripts.

SysVinit Runlevel	systemd Target	Meaning
0	poweroff.target	Shut down the system
1	rescue.target	Single-user/rescue mode
3	multi-user.target	Full multi-user, networking, no GUI
5	graphical.target	Multi-user + graphical login
6	reboot.target	Reboot the system

The Command

```
journalctl -b
journalctl -> query and display logs collected by systemd's journal (systemd-journald)

-b -> show logs only from the current Boot (negative offset, e.g. -b -1, for previous boot)
```

Tie-back

This ties directly to Section 2: firmware/bootloader stages run before the kernel/userspace split exists, and the moment PID 1 launches is exactly when the userspace layer comes into being. It also sets up systemd (Section 11) and kernel interfaces (Section 12), which only become inspectable after this sequence completes.

Key Takeaway

Firmware finds a bootloader -> GRUB2 loads kernel+initramfs -> kernel mounts the real root via initramfs -> kernel launches PID 1 (systemd), which starts every other service. initramfs exists to solve the bootstrapping problem of needing drivers to reach a filesystem that itself contains those drivers.

Section 3 Summary Table

Stage	What Happens	Key File/Tool
Firmware	POST, finds bootloader (BIOS:MBR; UEFI:ESP+Secure Boot)	--
Bootloader	GRUB2 loads kernel+initramfs, passes boot params	/etc/default/grub, /boot/grub/grub.cfg
Kernel init	Decompress kernel, detect hardware, mount real root	initramfs/initrd
Init takeover	PID 1 (systemd) starts all other services	journalctl -b

SECTION 4: Filesystem Hierarchy Standard (FHS)

4.1 What FHS Is and Why It Exists

FHS is a specification defining what top-level directories should exist on a Linux system and what content belongs in each. Without it, every distro could invent its own layout. FHS guarantees that regardless of distro, /etc holds configuration and /var holds variable runtime data, so scripts and muscle memory transfer everywhere.

4.2 The Directories

```
ls /
```

Directory	Purpose
/bin, /sbin	Essential binaries -- /bin for all users, /sbin traditionally for admin tools
/etc	Configuration files -- almost always plain text
/home	Per-user personal directories
/var	Variable data: logs, spool, cache
/usr	Installed programs & libraries (Unix System Resources, not 'user')
/tmp	Temporary files, typically cleared on reboot
/opt	Optional/third-party software
/proc	Virtual filesystem -- live kernel/process data [OS]
/sys	Virtual filesystem -- kernel device/driver data [OS]

/dev	Device files (block & character devices)
/lib, /lib64	Shared libraries

[OS] -> /proc and /sys are the clearest illustration of the kernel/userspace boundary: they are not real files backed by disk blocks -- they are a live, in-memory representation the kernel generates on demand, letting userspace inspect kernel state through the ordinary file-reading interface.

⚠ NOTE: A common point of confusion is /usr vs /home. /usr has nothing to do with individual user accounts -- it holds installed software (most of what apt/dnf install lands under /usr), while /home holds personal files belonging to human users.

4.3 Everything Is a File

The design decision underlying most of this structure is Unix's 'everything is a file' philosophy: regular documents, device hardware (/dev/sda), running kernel state (/proc), and network sockets are all represented as things you can open, read, and write using the same small set of syscalls (open, read, write, close). This is why a single tool like cat can meaningfully print a text file's contents and dump raw data from a device file -- both are just 'files' as far as the interface is concerned.

Comparison: Real Filesystem Directories vs Virtual (Kernel) Directories

Aspect	/etc, /home, /var, /usr	/proc, /sys
Backing storage	Actual disk blocks	None -- generated live by kernel in RAM
Persists across reboot	Yes	No -- regenerated fresh every boot
Meaningful df size?	Yes, real disk usage	No, reports 0 bytes
What it represents	Files created by user/software	Live kernel/process state at this instant

Tie-back

This extends Section 3: the initramfs stage mounts the real root filesystem -- the structure defined here -- and everything GRUB was searching for lives inside this FHS layout. It also sets up Section 5 (Disk & Filesystem Management) and previews Section 12, where /proc and /sys get their full deep-dive.

Key Takeaway

FHS gives every Linux distro a predictable, standardized layout, and /proc/sys are the two directories that break the 'backed by disk' assumption -- because they exist to expose live kernel state, not to store data.

Section 4 Summary Table

Directory	Type	Content
-----------	------	---------

/bin, /sbin	Disk-backed	Essential binaries
/etc	Disk-backed	Configuration files
/home	Disk-backed	Per-user personal directories
/var	Disk-backed	Logs, spool, cache
/usr	Disk-backed	Installed programs & libraries
/tmp	Disk-backed	Temporary files, cleared on reboot
/opt	Disk-backed	Third-party/optional software
/proc	Virtual (kernel)	Live process/kernel state
/sys	Virtual (kernel)	Device/driver object model
/dev	Special (device files)	Hardware device interfaces
/lib, /lib64	Disk-backed	Shared libraries

SECTION 5: Disk & Filesystem Management [GAP-FILLED ADDITION]

5.1 Why This Sits Here

Section 4 defined what FHS directories mean logically but left open what physical disk or partition actually backs / or /home. This section covers how Linux discovers block devices, mounts a filesystem onto the directory tree, and how usage is inspected at runtime.

5.2 Viewing Block Devices

```
lsblk
```

lsblk -> list block devices -- every disk/partition the kernel sees, tree structure, size, current mount point

[OS] -> this is a practical window into how the kernel represents storage hardware -- each entry corresponds to a device file under /dev, detected during boot and exposed before any filesystem was mounted.

5.3 Mounting: Attaching a Filesystem to the Tree

A raw block device is just a container of bytes until you mount it -- attach it at some path in the directory tree so its contents become accessible through ordinary file operations.

```
mount /dev/sdb1 /mnt/data
```

mount -> attach a filesystem to the directory tree

/dev/sdb1 -> the source block device/partition to mount

/mnt/data -> the target directory (mount point)

⚠ NOTE: Mounting this way is temporary -- it disappears on reboot unless also registered in /etc/fstab.

5.4 Persisting Mounts: /etc/fstab

/etc/fstab (filesystem table) is read by the kernel and systemd at boot to know which filesystems to mount automatically and where.

```
UUID=1234-5678 /boot ext4 defaults 0 2
/dev/sdb1 /mnt/data ext4 defaults 0 2
UUID=... or /dev/sdb1 -> source device -- UUID preferred, stable across reboots even if device enumeration order changes
```

/boot, /mnt/data -> the mount point (target directory)

ext4 -> the filesystem type

defaults -> mount options (rw, suid, dev, exec, auto, nouser, async)

0 -> dump utility flag (legacy, almost always 0 today)

2 -> fsck check order (0=don't check, 1=check first/root, 2=check after)

[OS] -> UUIDs defend against real nondeterminism -- device enumeration order is decided by the kernel at boot and is not guaranteed stable, while a UUID is baked in at filesystem creation and stays stable.

5.5 Checking Usage: df and du

```
df -h
du -sh /var/log
df -> 'disk free' -- usage per MOUNTED FILESYSTEM
```

du -sh -> 'disk usage' -- usage per FILE/DIRECTORY, summarized, human-readable

Comparison: df vs du

Aspect	df	du
Scope	Whole mounted filesystems	Specific files/directories
Answers	Is my disk full?	What's taking up space in here?
Speed	Instant (metadata)	Can be slow (walks every file)
Common use	Spotting a full partition	Hunting down space usage

⚠ NOTE: df and du can disagree: if a program has a file open that has since been rm'd, the kernel keeps disk space allocated until that process closes it -- so du shows less usage than df until the holding process exits.

Reference Pattern: fstab Field Order

```
GENERAL:  <device>    <mount point>    <fs type>    <options>    <dump>
<fsck order>

EXAMPLE 1: UUID=abcd-1234    /                ext4    defaults    0    1
EXAMPLE 2: UUID=5678-efgh    /boot            ext4    defaults    0    2
EXAMPLE 3: /dev/sdb1         /mnt/data        ext4    defaults,noatime 0    2
EXAMPLE 4: tmpfs             /tmp             tmpfs    defaults,size=2G 0    0
```

Tie-back

This completes Section 4: FHS gives the logical layout, and this section shows how it is physically assembled from block devices via mounting. It also previews Section 12, since `/proc/mounts` is a live kernel-exposed view of exactly this mount table.

Key Takeaway

A mount is not copying or moving data -- it redirects a path in the directory tree to read/write through to a specific underlying filesystem, and `/etc/fstab` is what makes that redirection survive a reboot.

Section 5 Summary Table

Command	Purpose
<code>lsblk</code>	List block devices and current mount points
<code>mount /dev/X /path</code>	Temporarily attach a filesystem
<code>/etc/fstab</code>	Config file for mounts persisting across reboot
<code>df -h</code>	Disk usage per mounted filesystem
<code>du -sh <dir></code>	Disk usage of a specific file/directory

SECTION 6: Navigation & File Operations

6.1 Navigation

```
pwd
cd /var/log
ls -lah
```

pwd -> print working directory

cd -> change directory

ls -> list directory contents

-l -> long format: permissions, owner, group, size, date

-a -> include hidden files (dotfiles)

-h -> human-readable sizes

-R -> recursive listing

6.2 Search

Command	Purpose	Speed
<code>find</code>	Search live filesystem by name/type/size/time/permissions	Slower -- walks directories
<code>locate</code>	Search a prebuilt filename index	Very fast -- only as current as last index update

updatedb	Manually rebuild the locate index	--
which	Full path of executable that would run for a command	Instant -- checks \$PATH
whereis	Binary, source, and man page locations	Instant

```
find /var/log -name "*.log" -mtime -7
find /var/log -> search starting from this directory

-name "*.log" -> match filenames against this glob pattern

-mtime -7 -> modified within the last 7 days
```

⚠ NOTE: locate can return files that no longer exist or miss recently created ones (cached index). find is always accurate but slower on large trees.

6.3 File Operations

```
cp -r source/ dest/
mv old_name new_name
rm -rf directory/
mkdir -p a/b/c
touch newfile.txt
cp -r -> copy files/directories recursively

mv -> move or rename

rm -rf -> remove recursively, forcing (no prompts)

mkdir -p -> create parent directories as needed, no error if they exist

touch -> create empty file or update its modification timestamp
```

⚠ NOTE: rm -rf is the single most dangerous everyday command on Linux -- no trash bin, no undo. A typo or an empty variable expansion (rm -rf \$VAR/ where \$VAR is empty) has destroyed production systems. Always double-check the path before running -r and -f together.

6.4 Links [OS: inodes]

Every file is actually two things: the data (bytes on disk) and the inode (a metadata record: permissions, owner, size, timestamps, and pointers to data blocks). A filename is just a directory entry pointing at an inode number -- this indirection is what makes both kinds of links possible.

```
ls -i file.txt
ln file.txt hardlink.txt
ln -s file.txt symlink.txt
ls -i -> show the inode number alongside each filename

ln file target -> create a HARD link -- a second directory entry pointing at the SAME inode

ln -s file target -> create a SYMBOLIC (soft) link -- a separate file whose content is a pathname
```

Hard links are additional names for the exact same inode -- the filesystem only frees the underlying data when the last hard link is removed. Hard links cannot cross filesystems and cannot point at directories. Symbolic links are a separate file whose content is literally a path string -- deleting the original leaves a broken/dangling symlink. Symlinks can cross filesystems and point at directories.

Comparison: Hard Links vs Symbolic Links

Aspect	Hard Link	Symbolic Link
What it points to	Same inode (same data)	A pathname (text string)
Survives original deleted?	Yes -- until last hard link removed	No -- becomes broken/dangling
Can cross filesystems?	No	Yes
Can link to a directory?	No	Yes
Appears as	Indistinguishable from 'original'	Distinct type, ls -l shows arrow to target
Command	ln file target	ln -s file target

6.5 Viewing & Editing

```
cat file.txt
tac file.txt
head -n 20 file.txt
tail -n 20 file.txt
tail -f /var/log/syslog
less file.txt
```

cat -> print entire file contents at once

tac -> print a file's lines in reverse order

head -n 20 -> show the first 20 lines

tail -n 20 -> show the last 20 lines

tail -f -> follow -- print new lines as they're appended, live

less -> pager -- scroll/search without loading the whole file into scrollbar

less vs more: both are pagers, but more is older and limited to scrolling forward only, and can struggle with piped input. less can scroll both directions, supports live search (/pattern), and does not need to read the entire file before displaying it.

vim is a modal editor -- keystrokes mean different things depending on mode. Normal mode (default): keystrokes are commands (h/j/k/l to move, dd delete a line, yy copy a line). Insert mode (enter with i): keystrokes are typed into the file. Visual mode (enter with v): select text to operate on.

```
:w (write/save)
:q (quit)
:wq (write then quit)
:q! (quit without saving)
/word (search forward)
:%s/old/new/g (find-and-replace globally)
```

⚠ NOTE: The most common vim beginner mistake is typing without pressing i first, causing keystrokes to be interpreted as commands. Esc returns to normal mode, from which :q! always exits safely.

Tie-back

This section is the hands-on toolkit for everything conceptual so far: the FHS/proc/sys directories from Section 4 are now literally explorable, and inodes here are the same mechanism behind the df/du disagreement noted in Section 5.

Key Takeaway

A filename is never the file itself -- it is a pointer to an inode. This explains hard links, why rm does not always free disk space immediately, and how the filesystem is structured underneath the directory tree.

Section 6 Summary Table

Category	Commands	Key Concept
Navigation	pwd, cd, ls -lah, ls -R	Moving around and listing the filesystem
Search	find, locate, updatedb, which, whereis	Live vs indexed search vs \$PATH lookup
File ops	cp -r, mv, rm -rf, mkdir -p, touch	Copy/move/delete/create
Links	ln, ln -s, ls -i	Filenames point to inodes
Viewing/Editing	cat, tac, head, tail -f, less, vim	Reading and editing content, live log following

SECTION 7: I/O Redirection & Pipes [GAP-FILLED ADDITION]

7.1 The Three Standard Streams

Every process is born with three open file descriptors already connected -- a direct consequence of the syscall interface and 'everything is a file' principles.

Stream	FD	Default destination	Purpose
stdin	0	Keyboard	Where a program reads input from
stdout	1	Terminal screen	Where a program writes normal output
stderr	2	Terminal screen	Where a program writes error/diagnostic messages

[OS] -> a file descriptor is a small integer the kernel uses as a process's handle to an open file/pipe/socket. Redirection is a process asking the kernel to change what a descriptor number points to, via the open/dup2 syscall family.

⚠ NOTE: stdout and stderr both default to the terminal, making them look identical -- but they are separate streams, and redirecting one does not redirect the other unless explicitly told to.

7.2 Redirection

```
command > output.txt      # stdout to file (overwrite)
command >> output.txt     # stdout to file (append)
command 2> errors.txt     # stderr to file
command > all.txt 2>&1    # BOTH stdout and stderr to same file
command < input.txt      # read stdin FROM a file
```

> -> redirect stdout to a file, OVERWRITING

>> -> redirect stdout to a file, APPENDING

2> -> redirect stderr (fd 2) to a file

2>&1 -> redirect stderr to wherever stdout (fd 1) currently points

< -> feed a file's contents in as stdin

⚠ NOTE: Order matters with 2>&1. 'command > all.txt 2>&1' correctly merges both into all.txt. But 'command 2>&1 > all.txt' does NOT merge them -- stderr is duplicated to the terminal (stdout's target at that moment), then stdout alone is redirected to the file. This ordering trap is a common shell scripting bug.

7.3 Pipes: Connecting Processes

A pipe connects the stdout of one process directly to the stdin of another, without touching disk - the Unix philosophy made concrete.

```
ps aux | grep nginx | wc -l
```

ps aux -> list all running processes

| **grep nginx** -> pipe stdout as stdin; filter lines containing 'nginx'

| **wc -l** -> pipe filtered output; count lines received

[OS] -> a pipe is a kernel-managed, unidirectional in-memory buffer with two file descriptor ends. Because it is in-memory rather than a temp file, data flows as it is produced and both processes can run concurrently.

7.4 tee: Splitting a Stream

```
some_command | tee output.txt | grep "error"
```

tee -> reads stdin, writes identical copies to BOTH stdout AND named file(s)

This is the standard pattern for both watching live output and saving it to a file, without running the command twice.

Comparison: Redirection Operators at a Glance

Operator	Direction	Target Stream	Overwrites/Appends
>	out	stdout (fd 1)	Overwrite
>>	out	stdout (fd 1)	Append
2>	out	stderr (fd 2)	Overwrite
2>>	out	stderr (fd 2)	Append
2>&1	out	stderr -> stdout's target	Inherits stdout's mode
<	in	stdin (fd 0)	--
	out->in	one process's stdout -> next's stdin	--

Reference Pattern: Common Redirection Combinations

```
EXAMPLE 1:  script.sh > log.txt 2>&1
            -> ALL output (normal + errors) into log.txt
EXAMPLE 2:  script.sh > log.txt 2> errors.txt
            -> normal output and errors in SEPARATE files
EXAMPLE 3:  script.sh > /dev/null 2>&1
            -> discard ALL output (common in cron jobs)
EXAMPLE 4:  long_command | tee full_log.txt | tail -20
            -> save everything, show only last 20 lines live
```

/dev/null is a special device file that silently discards anything written to it and returns end-of-file immediately if read -- the standard Unix idiom for 'discard this output entirely.'

Tie-back

This builds on Section 6: tail -f is a live-follow of a file, while command | tee | grep chains that idea across processes. File descriptors here resurface in Section 12 (/proc/<pid>/fd) and Section 13 (open/read/write/close syscalls are literally what's happening under every redirection and pipe).

Key Takeaway

stdout and stderr are separate streams by design, specifically so normal output and error output can be handled -- captured, filtered, or discarded -- independently.

Section 7 Summary Table

Concept	Symbol/Command	Purpose
Standard streams	stdin(0), stdout(1), stderr(2)	Default I/O channels for every process
Overwrite redirect	> , 2>	Send stdout/stderr to a file, replacing
Append redirect	>> , 2>>	Send stdout/stderr to a file, keeping existing
Merge streams	2>&1	Point stderr at stdout's current target

Input redirect	<	Feed a file in as stdin
Pipe		Connect one process's stdout to next's stdin
Tee	tee	Split a stream to screen and file simultaneously
Discard	/dev/null	Special device file that discards all writes

SECTION 8: Permissions & Ownership [GAP-FILLED ADDITION]

8.1 The Permission Model

Every file and directory carries three permission sets, one each for owner, group, and others -- the practical, everyday face of privilege separation applied at the filesystem level.

```
ls -l file.txt
-rwxr-xr-- 1 allen developers 4096 Sep 15 10:00 file.txt
- -> file type (- = regular file, d = directory, l = symlink)
```

rwx -> OWNER permissions: read, write, execute

r-x -> GROUP permissions: read, execute

r-- -> OTHERS permissions: read only

[SEC] -> this is discretionary access control -- every file has an owner who decides who else can read/write/execute it, enforced by the kernel on every access attempt as part of the open() syscall path.

Permission	On a File	On a Directory
r (read)	View file contents	List directory contents (ls)
w (write)	Modify file contents	Create/delete/rename entries inside it
x (execute)	Run the file as a program/script	cd into it / traverse through it

⚠ NOTE: The directory-execute permission is the most common source of confusion. A directory with r but not x lets you see filenames (ls works) but not access them (cat fails) -- x on a directory means 'can traverse into it', a different meaning than x on a regular file.

8.2 chmod: Changing Permissions

```
chmod u+x script.sh
chmod 755 script.sh
chmod -> change mode -- modify permission bits
```

u+x -> symbolic: u/g/o/a target, +/-/= add/remove/set, then permission letter

755 -> octal: sum of r=4,w=2,x=1 per owner/group/others digit

r = 4 w = 2 x = 1

7 = rwx (4+2+1) 6 = rw- (4+2) 5 = r-x (4+1)

4 = r-- (4) 3 = -wx (2+1) 2 = -w- (2)

1 = --x (1) 0 = --- (nothing)

755 -> owner: 7 (rwx) group: 5 (r-x) others: 5 (r-x)

644 -> owner: 6 (rw-) group: 4 (r--) others: 4 (r--)

8.3 chown: Changing Ownership

```
chown allen:developers file.txt
```

```
chown -R allen:developers /var/www/myapp
```

chown -> change owner -- new owning user and/or group

-R -> recursive -- apply to a directory and everything inside it

⚠ NOTE: chown normally requires root/sudo privileges -- a regular user cannot arbitrarily give away ownership of their own files, which would dodge disk quota accounting or dump files into another user's space.

8.4 umask: The Default Permission Mask

New files/directories start from a base (666 for files, 777 for directories) and have the umask subtracted from that base.

```
umask
umask 022
```

With the common default umask 022: new files land at 666-022=644 (rw-r--r--), new directories at 777-022=755 (rwxr-xr-x). A freshly created file is never executable by default because the file-creation base (666) never included execute in the first place -- umask only ever removes from what the base already grants.

8.5 Special Permission Bits

Bit	Symbol	Effect
SUID	s in owner's execute slot	Program runs with the FILE OWNER's privileges, not the invoking user's (e.g. /usr/bin/passwd)
SGID	s in group's execute slot	New files in a directory inherit the directory's group
Sticky bit	t in others' execute slot	Only the file's owner (or root) can delete/rename files inside, even if others have write (e.g. /tmp)

[SEC] -> SUID is a well-known privilege-escalation risk -- a buggy or trickable SUID root binary can let an unprivileged user execute code with root privileges, which is why audits routinely run `find / -perm -4000` to list all SUID binaries.

Comparison: chmod Symbolic vs Octal Notation

Aspect	Symbolic (u+x)	Octal (755)
Readability	Easier for a single change	Requires mental math
Use case	Adjust one permission without affecting others	Set the full permission set at once
Example	chmod g-w file	chmod 644 file

Reference Pattern: Common chmod Values

```
GENERAL:      chmod <owner><group><others> file

644  (rw-r--r--) -> typical for regular files
755  (rwxr-xr-x) -> typical for scripts/executables and directories
600  (rw-----) -> private files -- SSH keys, sensitive configs
700  (rwx-----) -> private directories -- owner-only
777  (rwxrwxrwx) -> everyone can do everything -- almost always a
mistake
```

⚠ NOTE: chmod 777 is a common 'quick fix' but rarely correct -- anyone on the system can read/modify/execute that file. The right fix is usually identifying the correct owner/group via chown and a narrower permission like 644 or 755.

Tie-back

This connects to Section 6's inode concept: permissions, owner, and group are stored in the inode itself, which is why a hard link always shows identical permissions to its 'original' -- they are the same inode. It sets up Section 9 (Users & Groups), since 'owner' and 'group' are meaningless without understanding what a user/group actually is.

Key Takeaway

Execute on a directory means 'can traverse into it,' not 'can run it' -- the single most-missed nuance in the permission model.

Section 8 Summary Table

Command/Concept	Purpose
ls -l	View permissions, owner, group
chmod (symbolic/octal)	Change permission bits
chown user:group	Change owner and/or group
umask	Default permission mask for new files/dirs
SUID	Run as file owner's privileges
SGID	New files inherit directory's group
Sticky bit	Only owner can delete own files in shared dir

SECTION 9: Users & Groups [GAP-FILLED ADDITION]

9.1 What a User Actually Is

A user is a numeric ID (UID) that the kernel attaches to every process that user starts, and that same UID is checked against a file's owner field during every permission check from Section 8.

```
cat /etc/passwd
allen:x:1000:1000:Allen Dev:/home/allen:/bin/bash
allen -> username
```

x -> placeholder -- actual password hash lives in /etc/shadow

1000 -> UID (user ID)

1000 -> GID (primary group ID)

Allen Dev -> GECOS field -- free-text comment, usually full name

/home/allen -> home directory

/bin/bash -> login shell

⚠ NOTE: /etc/passwd is world-readable by design so tools like `ls -l` can translate a numeric UID back into a username. The password hash was deliberately moved to /etc/shadow (root-only readable) once offline brute-forcing of a world-readable password file became a practical risk.

9.2 UID Ranges Matter

```
UID 0      -> root -- the superuser, bypasses virtually all permission
checks
UID 1-999  -> system/service accounts (www-data, mysql, etc.) -- not
for human login
UID 1000+  -> regular human user accounts (distro-dependent start; some
use 500)
```

[SEC] -> giving each service its own low UID (rather than running as root) is the principle of least privilege in practice -- if a compromised web server process runs as `www-data`, the attacker inherits only that limited identity, not root.

9.3 Groups

```
cat /etc/group
groups allen
/etc/group -> defines groups and membership
```

groups allen -> shows every group a given user belongs to

Every user has exactly one primary group (from /etc/passwd's GID field) and can belong to any number of secondary/supplementary groups (/etc/group) -- how you grant extra access (e.g. a docker group) without changing primary identity.

9.4 Managing Users and Groups

```

useradd -m -s /bin/bash newuser
usermod -aG sudo newuser
passwd newuser
userdel -r newuser
groupadd developers
useradd -m -s ... -> -m creates home directory, -s sets login shell

```

usermod -aG sudo -> -a APPENDS to the group WITHOUT removing existing memberships

passwd newuser -> set/change password

userdel -r -> delete user; -r also removes home directory and mail spool

groupadd -> create a new group

⚠ NOTE: The most common usermod mistake is forgetting -a. 'usermod -G sudo newuser' (no -a) REPLACES the entire supplementary group list with just sudo, silently removing all other memberships. Always use -aG.

9.5 sudo vs su

Aspect	su	sudo
What it does	Switches to another user's full session, needs THAT user's password	Runs a single command as another user, needs YOUR OWN password
Logging	Minimal	Every invocation logged (full audit trail)
Granularity	All-or-nothing shell access	Configurable per-user/per-command via /etc/sudoers
Typical use	Rare on modern systems	Standard day-to-day privileged commands

[SEC] -> sudo supports both auditability and least privilege far better than handing out the root password -- which is why most modern distros disable direct root login and route all privileged access through sudo.

Reference Pattern: /etc/passwd and /etc/group Field Structure

```

/etc/passwd:  username : x : UID : GID : comment : home_dir : shell
/etc/group:   groupname : x : GID : comma,separated,member,list

```

```

EXAMPLE (/etc/passwd):
  allen:x:1000:1000:Allen Dev:/home/allen:/bin/bash
  www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin

```

```

EXAMPLE (/etc/group):
  sudo:x:27:allen
  developers:x:1001:allen,priya

```

/usr/sbin/nologin is the conventional shell for service accounts, preventing anyone from logging into them interactively even if they obtained the credentials.

Tie-back

This is the direct missing link Section 8 assumed existed: 'owner'/'group' in `ls -l` output is the human-readable translation of the numeric UID/GID resolved via `/etc/passwd` and `/etc/group`. It connects forward to Section 14, since a process's effective permissions come from the UID/GID of the launching user.

Key Takeaway

UID 0 is root; system services get their own low UIDs to run with least privilege instead of as root; sudo is preferred over su because it is per-command, per-user, and fully audit-logged.

Section 9 Summary Table

Command/File	Purpose
<code>/etc/passwd</code>	User account records (UID, GID, home, shell)
<code>/etc/shadow</code>	Password hashes (root-only readable)
<code>/etc/group</code>	Group definitions and membership
<code>useradd / userdel</code>	Create / delete a user
<code>usermod -aG</code>	Add a user to a supplementary group (append)
<code>passwd</code>	Set/change a password
<code>groupadd</code>	Create a new group
<code>sudo vs su</code>	Per-command audited privilege vs full session switch

SECTION 10: Package Management

10.1 Why Package Managers Exist

Before package managers, installing software meant manually downloading source, resolving every dependency, compiling, and hoping nothing conflicted. A package manager automates this: tracking what's installed, what version, what each package depends on, and how to cleanly add/remove software.

10.2 Debian Family: apt and dpkg

```
sudo apt update
sudo apt install nginx
sudo apt remove nginx
sudo apt upgrade
```

apt update -> refresh the LOCAL index of available packages -- does not install anything

apt install -> install a package and its dependencies

apt remove -> uninstall, leaving config files behind

apt upgrade -> upgrade all installed packages

apt is the high-level tool understanding dependencies and repositories. dpkg is the low-level tool that unpacks/installs a .deb file with no automatic dependency fetching.

```
sudo dpkg -i package.deb
dpkg -i -> install a LOCAL .deb; fails on missing dependencies
```

⚠ NOTE: A common workflow after dpkg -i fails on missing deps: run 'sudo apt install -f' immediately after -- the -f (fix broken) flag tells apt to find and install whatever dependencies the manually-installed package needs.

10.3 RHEL Family: dnf/yum and rpm

```
sudo dnf install nginx
sudo dnf remove nginx
sudo dnf update
```

yum is the predecessor to dnf, replaced starting with Fedora 22 / RHEL 8 with a faster dependency resolver; syntax is nearly identical and yum is often aliased to dnf today. rpm is the low-level tool, exactly analogous to dpkg.

```
sudo rpm -ivh package.rpm
rpm -ivh -> -i install, -v verbose, -h hash-mark progress bar
```

Comparison: Debian vs RHEL Package Tooling

Layer	Debian Family	RHEL Family
Package format	.deb	.rpm
High-level (deps+repos)	apt	dnf (modern) / yum (legacy)
Low-level (local file only)	dpkg	rpm
Fix broken local install	apt install -f	dnf install (usually auto-resolves)

10.4 Dependency Resolution

When you run apt install nginx, the package manager reads nginx's declared dependencies, checks what's satisfied, and builds a full install plan for everything missing, in the correct order. This is a non-trivial computational problem in general (version constraints, conflicts, multiple providers of the same capability), which is why dnf's newer resolver was a significant improvement over yum's.

10.5 Repositories & GPG Signing

A repository is a remote server hosting packages and an index of what's available.

```
cat /etc/apt/sources.list
deb http://archive.ubuntu.com/ubuntu jammy main restricted
deb -> binary package repository (vs deb-src for source packages)

jammy -> distro release codename this repo serves

main restricted -> which repository components to include
```

[SEC] -> every package in a properly configured repo is GPG-signed, and the package manager verifies the signature before installing -- closing an attack surface where an intercepted connection or compromised mirror could substitute a malicious package. This mirrors the Secure Boot trust chain from Section 3, applied at the package level.

⚠ NOTE: If apt update reports an NO_PUBKEY or unsigned-repository error, the correct fix is importing that repository's real GPG key from a trusted source -- never disabling signature verification globally.

10.6 Compiling From Source

```
./configure
make
sudo make install
```

`./configure` -> checks the system for required libraries/compilers, generates a tailored Makefile

`make` -> compiles source code into binaries per the Makefile

`make install` -> copies compiled binaries/libraries to final system locations (often /usr/local)

⚠ NOTE: Software installed via make install is invisible to apt/dnf -- no package record, no clean uninstall, no automatic security updates. Package managers are strongly preferred whenever a package is available; compiling from source should be a last resort.

Reference Pattern: Package Manager Command Equivalents

ACTION	DEBIAN/UBUNTU (apt)	RHEL/FEDORA (dnf)
Refresh index	apt update	(dnf does this automatically)
Install	apt install <pkg>	dnf install <pkg>
Remove	apt remove <pkg>	dnf remove <pkg>
Upgrade all	apt upgrade	dnf update
Search	apt search <term>	dnf search <term>
Show info	apt show <pkg>	dnf info <pkg>
Local file install	dpkg -i <file>.deb	rpm -ivh <file>.rpm

Tie-back

This connects back to Section 1: the chosen distro family determines every command here. It also connects to Section 5 -- /usr/local (where make install typically places software) and repository caches under /var are both FHS locations already covered. The GPG-signing chain here parallels Secure Boot from Section 3.

Key Takeaway

apt/dnf are high-level tools resolving dependencies and fetching from repositories, while dpkg/rpm are low-level tools that just install a local package file with no dependency resolution - and repository packages are GPG-signed so the manager can verify they have not been tampered with. Compiling from source bypasses the package manager's tracking entirely, so it should be a last resort.

Section 10 Summary Table

Concept	Debian	RHEL	Purpose
High-level manager	apt	dnf/yum	Install with dependency+repo resolution
Low-level manager	dpkg	rpm	Install a local package file directly
Package format	.deb	.rpm	--
Repository config	/etc/apt/sources.list	/etc/yum.repos.d/	Where package sources are defined
Signing	GPG [SEC]	GPG [SEC]	Verifies package authenticity
From source	configure/make/make install	same	Last-resort untracked install

SECTION 11: systemd Deep Dive

11.1 Unit Types

Everything systemd manages is represented as a unit, a configuration file describing something systemd should start, stop, or track.

Unit Type	Suffix	Manages
Service	.service	A long-running process or a one-shot task
Socket	.socket	A network/IPC socket -- can trigger service start on first connection
Timer	.timer	A scheduled trigger -- systemd's alternative to cron
Target	.target	A synchronization point grouping other units (multi-user.target, etc.)
Mount	.mount	A filesystem mount point, integrated into systemd's dependency graph

11.2 systemctl: Controlling Units

```
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx
sudo systemctl enable nginx
sudo systemctl disable nginx
systemctl status nginx
```

start -> run the unit NOW, this boot session only

stop -> stop the unit NOW

restart -> stop then start

enable -> symlink so the unit auto-starts on future boots (does not start now)

disable -> remove that symlink (does not stop current run)

status -> current run state, recent logs, enabled status

⚠ NOTE: start and enable are independent axes. 'systemctl start nginx' runs it now but won't survive reboot. 'systemctl enable nginx' sets it to auto-start next boot but does nothing to current state. In production you almost always want both -- 'enable --now' does both in one command.

11.3 journalctl: Querying Logs

```
journalctl -u nginx
journalctl -u nginx --since "1 hour ago"
journalctl -p err
journalctl -f
```

-u nginx -> filter logs to just this unit

--since "1 hour ago" -> filter by time range (also --until)

-p err -> filter by priority level, this and more severe

-f -> follow -- live-stream new entries

[OS] -> journalctl reads from systemd-journald's binary log storage rather than plain text under /var/log -- structured, indexed storage lets you filter by unit/priority/boot/time far more efficiently than grep-ing text logs.

11.4 Writing a Custom .service File

```
[Unit]
Description=My Custom App
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure
User=myappuser

[Install]
WantedBy=multi-user.target
```

[Unit] -> metadata and ordering/dependency info

Description= -> human-readable label shown in systemctl status

After=network.target -> ORDERING only -- start after networking, does not force it to be pulled in

[Service] -> how to run and supervise the process

ExecStart= -> exact command systemd runs to start the service

Restart=on-failure -> auto-restart on non-zero exit, not on clean exit

`User=myappuser` -> run as this user instead of root -- least-privilege from Section 9

`[Install]` -> how the unit integrates with enable

`WantedBy=multi-user.target` -> becomes part of what starts when this target is reached

⚠ NOTE: After= controls ordering only, not dependency. To make a unit require another to be running, add Requires= as well -- After= alone says nothing about what happens if the other unit never starts.

11.5 Targets vs Runlevels -- Revisited

A target is itself a unit, often with no content beyond being a name other units declare `WantedBy=`. When systemd 'reaches' `multi-user.target`, it starts every unit that declared itself `WantedBy=multi-user.target` -- activated when `systemctl enable` was run.

11.6 systemd Timers vs Cron

```
systemctl list-timers
# myjob.timer
[Timer]
OnCalendar=daily
Persistent=true

[Install]
WantedBy=timers.target
```

`OnCalendar=daily` -> run once per day (flexible calendar expressions supported)

`Persistent=true` -> run missed jobs ASAP on next boot if the machine was off, instead of skipping

Comparison: systemd Timers vs Cron

Aspect	Cron	systemd Timers
Logging	Depends on script (often none)	Automatic, unified via <code>journalctl -u</code>
Missed-run handling	No built-in catch-up	<code>Persistent=true</code> runs missed jobs on next boot
Dependency awareness	None	Can depend on other units (network, mounts)
Syntax	Compact crontab line	More verbose <code>.timer</code> unit file
Config location	<code>crontab -e</code> , <code>/etc/cron.d/</code>	Paired <code>.timer</code> + <code>.service</code> files

⚠ NOTE: Despite systemd timers being more capable, cron remains extremely common - its syntax is compact and universally recognized. Know both; check `systemctl list-timers` and `crontab -l /etc/cron.d/` before assuming which a system uses.

Reference Pattern: Common [Service] Directives

```
ExecStart=/path/to/binary --flag -> the command that starts the service
Restart=on-failure -> restart only on non-zero exit
Restart=always -> restart no matter how it exited
User=serviceuser -> run as a non-root user
```

```
WorkingDirectory=/opt/myapp      -> set working directory before
ExecStart
Environment=KEY=value           -> set an environment variable
```

Tie-back

This continues Section 3: PID 1 is systemd, and WantedBy= is how boot-time handoff gets implemented. User= is Section 9's least-privilege principle applied to how a service runs, and .mount units connect back to Section 5's /etc/fstab.

Key Takeaway

[Unit] handles metadata/ordering, [Service] defines how the process runs and is supervised, [Install] defines what target pulls the unit in when enabled. enable and start are independent -- enabling wires a unit into boot activation, starting runs it now, and production deployments need both.

Section 11 Summary Table

Concept	Command/Directive	Purpose
Unit control	systemctl start/stop/restart	Immediate runtime control
Boot persistence	systemctl enable/disable	Whether a unit auto-starts on boot
Log querying	journalctl -u/-p/-f/--since	Filter and follow centralized logs
Service file	[Unit]/[Service]/[Install]	Metadata / execution / boot-integration
Scheduled tasks	.timer + .service pair	Modern, logged alternative to cron

SECTION 12: Kernel Interfaces to Userspace [OS]

12.1 Why These Filesystems Exist

Instead of needing a completely separate API for every piece of internal kernel state, the kernel reuses the same open/read/write interface every other file uses. Any tool that reads a text file -- cat, a script, a monitoring agent -- can inspect live kernel/process state with zero special-purpose code.

12.2 /proc: Process and Kernel State

```
cat /proc/cpuinfo
cat /proc/meminfo
cat /proc/version
/proc/cpuinfo -> detailed info on every logical CPU core

/proc/meminfo -> live memory statistics: total, free, buffers/cache, swap

/proc/version -> kernel version string, compiler, build date

ls /proc/$$/
$$ -> shell special variable holding the current shell's own PID
```

File	Reveals
status	Process state, memory usage, UID/GID, thread count
fd/	Every open file descriptor this process currently holds
maps	Virtual memory layout -- mapped regions and what backs them
cmdline	Exact command line the process was launched with
environ	The process's environment variables at launch
cwd	Symlink to the process's current working directory
exe	Symlink to the actual binary being executed

△ NOTE: `/proc/<pid>/fd/` directly demonstrates the file descriptor concept: `ls -l` on it shows numbered symlinks (0,1,2, and any others) pointing at exactly what each descriptor refers to -- confirming a file descriptor is a real, inspectable per-process table the kernel maintains.

12.3 /proc/sys: The Sysctl-Backed Bridge

A specific subtree, `/proc/sys/`, is different: many files are writable, and writing changes live kernel behavior immediately, without a reboot.

```
cat /proc/sys/net/ipv4/ip_forward
echo 1 > /proc/sys/net/ipv4/ip_forward
cat ...ip_forward -> reads current value (0=disabled, 1=enabled) of IP packet forwarding

echo 1 > ... -> WRITES a new value, changing kernel behavior immediately and live
```

12.4 /sys: Device and Driver Hierarchy

`/sys` is oriented specifically around the kernel object model -- devices, drivers, and buses, and how they relate.

```
ls /sys/class/net/
cat /sys/class/net/eth0/address
/sys/class/net/ -> every network interface the kernel recognizes

eth0/address -> MAC address of eth0, exposed as a readable kernel attribute
```

This extends Section 5's `lsblk`: block devices there are also represented under `/sys/block/`, and `udev` watches `/sys` for kernel-generated device events to create/remove `/dev` entries.

Comparison: /proc vs /sys

Aspect	/proc	/sys
Primary focus	Processes + general kernel state	Devices, drivers, kernel object model
Historical origin	Older, predates /sys	Introduced later (2.6 kernel) for devices
Typical use	<code>cat /proc/meminfo</code> , inspecting a PID	<code>ls /sys/class/net</code> , driver attributes
Writable subset	<code>/proc/sys/*</code> (sysctl-backed)	Many device/driver attribute files

12.5 sysctl: The Proper Interface to /proc/sys

```
sysctl net.ipv4.ip_forward
sudo sysctl -w net.ipv4.ip_forward=1
sysctl -a
sysctl net.ipv4.ip_forward -> READ current value -- dotted notation maps to the /proc/sys path

sysctl -w ...=1 -> WRITE a new value with proper validation vs a silent echo failure

sysctl -a -> list every currently readable kernel parameter

# /etc/sysctl.conf or /etc/sysctl.d/99-custom.conf
net.ipv4.ip_forward = 1
sysctl -p -> reload settings from sysctl.conf/sysctl.d and apply immediately
```

[PERF] -> runtime kernel tuning via sysctl is a direct lever on OS-level performance -- parameters like *vm.swappiness* or *net.core.somaxconn* let you tune scheduler, memory, and network stack behavior without recompiling or rebooting.

⚠ NOTE: Writing directly to */proc/sys/** with echo and using **sysctl -w** have the same effect on the running kernel, but **sysctl -w** validates input and gives clear errors, while a bad echo can fail silently. Prefer **sysctl** in scripts.

Reference Pattern: sysctl Dotted Notation <-> /proc/sys Path

```
GENERAL:          /proc/sys/<a>/<b>/<c>    <->   sysctl name: <a>.<b>.<c>

EXAMPLE 1:  /proc/sys/net/ipv4/ip_forward    <->   net.ipv4.ip_forward
EXAMPLE 2:  /proc/sys/vm/swappiness           <->   vm.swappiness
EXAMPLE 3:  /proc/sys/kernel/hostname         <->   kernel.hostname
EXAMPLE 4:  /proc/sys/fs/file-max             <->   fs.file-max
```

Tie-back

This fulfills Section 4's promise that */proc* and */sys* are live, kernel-generated views rather than disk storage. */proc/<pid>/fd* makes Section 7's file descriptors concretely inspectable, */proc/sys* and **sysctl** give the runtime handle on kernel behavior gated by Section 2's *syscall* boundary, and */sys/class/net* extends Section 5's device/mount picture.

Key Takeaway

These are not files backed by disk -- the kernel dynamically generates a readable/writable view of its own internal state, which is why they report 0 bytes in *df* but contain real, live data when you *cat* them.

Section 12 Summary Table

Interface	Focus	Writable?	Key Tool
<i>/proc</i>	Process + general kernel state	Read-only (except <i>/proc/sys</i>)	<i>cat /proc/meminfo</i>

/proc/<pid>/	Per-process live data	Read-only	status, fd/, maps, cmdline
/proc/sys	Tunable kernel parameters	Yes (root)	echo/sysctl -w
/sys	Device/driver object model	Some attributes writable	ls /sys/class/net
sysctl	Structured interface to /proc/sys	Yes (root)	sysctl -a / -w

SECTION 13: System Call Interface [OS: syscall interface, mode switching]

13.1 What a Syscall Actually Is

A system call is the only mechanism by which a userspace program (ring 3) requests a kernel service (ring 0). It's a controlled, deliberate mode switch, not a normal function call, even though it often looks like one to the programmer.

The program never chooses where execution jumps inside the kernel. It issues a special CPU instruction (syscall on x86-64), and the CPU hardware forces execution to a single, kernel-defined entry point. The kernel reads a syscall number (placed in a specific register before the instruction executes) and dispatches to the correct internal function via a lookup table (the syscall table).

[OS] -> this entire mechanism is 'mode switching' -- every syscall, even trivial ones like getpid(), involves a full ring 3 to ring 0 transition and back, with real measurable overhead versus an ordinary function call. This is why performance-sensitive software minimizes syscall frequency, e.g. buffering many small writes into one large write() call.

13.2 Common Syscall Categories

Category	Example Syscalls	Purpose
Process control	fork, exec, exit, wait	Creating, replacing, terminating, reaping processes
File management	open, read, write, close	Section 7's redirection/pipes built on these
Device management	ioctl	Device-specific operations not fitting read/write
Info maintenance	getpid, alarm	Querying identity, scheduling a timed signal

fork/exec is the two-step process behind every shell command: bash calls fork() to create a near-identical child, and that child calls exec() to replace its own memory image with the target program. fork duplicates a process, exec replaces what a process is running -- deliberately separate so a shell can set up redirection in the child before exec overwrites it.

13.3 Tracing Tools

```
strace ls
strace -f -e trace=open,openat ls
ltrace ls
```

strace -> trace every SYSCALL a process makes

-f -> follow forked child processes too

-e trace=open,openat -> restrict tracing to specific syscalls

ltrace -> trace LIBRARY calls (glibc functions), above the syscall layer

Comparison: strace vs ltrace

Aspect	strace	ltrace
What it traces	Kernel syscalls (ring 3->0 boundary)	Userspace library function calls
Layer	Below libc -- the real kernel interface	Above/at libc -- may never touch the kernel
Use case	Why can't this program open this file/port	What library functions is this program calling
Overhead	Significant slowdown	Significant, less precise for kernel-level debugging

13.4 Reading strace Output

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/nonexistent/file", O_RDONLY) = -1 ENOENT (No such file or directory)
```

openat(...) -> syscall name and arguments it was called with

= 3 -> RETURN VALUE -- successful open returns the new file descriptor number

= -1 ENOENT -> FAILED syscall -- -1 signals failure, errno explains why

[OS] -> every syscall's return value follows this convention: non-negative means success, -1 universally means failure with errno set -- which is exactly what makes strace broadly useful without special-case parsing per syscall.

⚠ NOTE: A strong interview answer to 'how would you debug why a program can't access a file': run **strace -e trace=open,openat,stat** and read down the list of **ENOENT** or **EACCES** failures, which usually points directly at the misconfigured path or permission without reading the program's source.

Reference Pattern: Syscall Return Value Convention

```
GENERAL:    success -> non-negative value (often meaningful: fd, byte count, PID)
            failure -> -1, with errno set to a specific reason code
```

```

EXAMPLE 1: open("file", O_RDONLY)      success -> 3 (a new file descriptor)
EXAMPLE 2: open("missing", O_RDONLY)  failure -> -1, errno = ENOENT
EXAMPLE 3: read(fd, buf, 100)         success -> bytes actually read (may
be < 100)
EXAMPLE 4: fork()                     success -> 0 in child, child's PID
in parent

```

Tie-back

This is the mechanical payoff of Section 2's kernel/userspace boundary and resolves how a syscall number resolves to kernel code. It retroactively explains Section 7 (every redirect/pipe is open/dup2/read/write underneath) and Section 12 (/proc/<pid>/fd is the kernel's live record of what open() has produced).

Key Takeaway

A syscall is the only way userspace can request a kernel service -- a controlled ring 3-to-ring 0 transition where the CPU jumps to a fixed kernel entry point, dispatched by syscall number. Every syscall follows the same return convention -- non-negative is success, -1 is failure with errno set -- which is exactly what makes strace a universal debugging tool.

Section 13 Summary Table

Category	Example Syscalls	Tool to Observe
Process control	fork, exec, exit, wait	strace -f
File management	open, read, write, close	strace -e trace=open,read,write
Device management	ioctl	strace -e trace=ioctl
Info maintenance	getpid, alarm	strace -e trace=getpid
Return convention	success>=0, failure=-1+errno	Reading raw strace output

SECTION 14: Process Management & Signals [GAP-FILLED ADDITION]

14.1 What a Process Is, Practically

Every process has a PID and a PPID (parent process ID) -- since fork() always creates a child from an existing process, every process except PID 1 has a parent, forming a tree.

```

ps aux
ps -ef --forest
ps aux -> a=all users, u=user-oriented format (CPU%/MEM%), x=include daemons without a terminal

ps -ef --forest -> -e every process, -f full-format (shows PPID), --forest renders the tree visually

```

Column	Meaning
PID	Process ID
%CPU	Percentage of CPU time currently consumed
%MEM	Percentage of physical RAM used

STAT	Process state (see below)
COMMAND	Command line the process was launched with

14.2 Process States

```

R -> Running or runnable
S -> Sleeping (interruptible) -- waiting on an event, can be woken by a
    signal
D -> Uninterruptible sleep -- usually waiting on disk I/O; CANNOT be
    killed normally
Z -> Zombie -- exited, but parent hasn't called wait() to collect its
    exit status
T -> Stopped -- suspended, typically via Ctrl+Z or SIGSTOP

```

[OS] -> the zombie state is a direct consequence of the fork/wait syscall pair -- a child's exit status must be stored somewhere until the parent retrieves it, and that somewhere is a zombie entry, consuming a PID slot but no memory or CPU, cleared only by wait() (or reparenting to PID 1 if the parent dies first).

⚠ NOTE: A process stuck in D state is a genuinely different problem from a merely slow one -- it cannot be killed with a normal signal, since the kernel won't deliver signals mid-way through certain low-level I/O. Many D-state processes usually point to a hardware/storage-layer problem, not application misbehavior.

14.3 Live Monitoring: top / htop

top is available by default on virtually every system, showing continuously refreshing system load and per-process usage. htop is a more modern, interactive replacement -- color output, mouse support, scrollable tree, easier process management -- but must be installed separately.

14.4 Signals

A signal is a limited, asynchronous form of inter-process communication -- a small integer sent to a process to notify it of an event or request an action, delivered regardless of what the process is doing (barring the D-state exception).

```

kill -15 1234
kill -9 1234
kill -1

```

kill -15 1234 -> SIGTERM -- please terminate gracefully; process can catch and clean up

kill -9 1234 -> SIGKILL -- immediate, unconditional termination by the KERNEL, cannot be caught

kill -1 -> list every available signal name/number

Signal	Number	Default Effect	Can Be Caught/Ignored?
SIGTERM	15	Graceful termination request	Yes
SIGKILL	9	Immediate termination	No -- kernel-enforced
SIGINT	2	Interrupt (Ctrl+C)	Yes

SIGSTOP	19	Suspend the process	No
SIGCONT	18	Resume a stopped process	Yes
SIGHUP	1	Originally 'terminal hung up'; often means 'reload config'	Yes

⚠ NOTE: kill -9 is frequently reached for as a first resort, but should be a last resort -- SIGKILL bypasses the process entirely, so it never gets to flush buffered writes, close connections cleanly, or remove lock files. Always try plain kill (SIGTERM, the default) first.

14.5 Job Control

```
long_running_command &
jobs
fg %1
bg %1
```

command & -> run in the BACKGROUND -- shell returns immediately

jobs -> list background/stopped jobs, numbered

fg %1 -> bring job 1 to the FOREGROUND

bg %1 -> resume a STOPPED job in the background

Ctrl+Z suspends the current foreground job (sends SIGSTOP, moving it to T state) without killing it. bg resumes it in the background, fg brings it back to the foreground.

14.6 Priority: nice and renice

```
nice -n 10 long_command
renice -n 5 -p 1234
```

nice -n 10 -> START a new process with niceness 10 (lower priority)

renice -n 5 -p 1234 -> CHANGE the niceness of an already-running process

Niceness ranges from -20 (highest priority) to +19 (lowest priority) -- a higher niceness number means the process is 'nicer' to everyone else, i.e. lower actual scheduling priority.

⚠ NOTE: Setting a negative niceness (higher priority) requires root -- an unprivileged user can only make their own processes nicer (increase the number), preventing a regular user from starving the system's CPU.

Reference Pattern: Signal-Sending Syntax

```
GENERAL:      kill -<signal number or name> <PID>
```

```
EXAMPLE 1:    kill -15 1234          (or just: kill 1234)    -> graceful
SIGTERM
```

```

EXAMPLE 2:      kill -9 1234                                -> forceful
SIGKILL
EXAMPLE 3:      kill -SIGHUP 1234                            -> same as:
kill -1 1234
EXAMPLE 4:      pkill -f "python.*myscript"                  -> kill by
command-line pattern

```

Tie-back

This is the practical layer on Section 13: fork/exec/wait are the syscalls behind everything here, ps reads the same data exposed under Section 12's /proc/<pid>/, and kill is a thin wrapper around the kill() syscall. It also connects to Section 11: systemctl restart and Restart=on-failure internally send exactly these signals to manage a service's lifecycle.

Key Takeaway

Always try SIGTERM (plain kill) first so a process can clean up gracefully, and only escalate to SIGKILL (kill -9) if it doesn't respond. A zombie process isn't a bug -- it's an already-dead process whose exit status the kernel holds until the parent calls wait(), costing almost no resources, unlike a runaway D-state or CPU-hogging process.

Section 14 Summary Table

Concept	Command	Purpose
Process snapshot	ps aux, ps -ef --forest	List/inspect running processes
Process states	R/S/D/Z/T	Running, sleeping, uninterruptible I/O, zombie, stopped
Live monitoring	top, htop	Continuously refreshing resource usage
Graceful stop	kill -15 (SIGTERM)	Ask a process to exit cleanly
Forceful stop	kill -9 (SIGKILL)	Kernel-enforced immediate termination
Job control	&, jobs, fg, bg, Ctrl+Z	Manage foreground/background jobs
Priority	nice, renice	Influence scheduler priority (-20 to +19)

SECTION 15: Kernel Modules [OS: kernel extensibility]

15.1 Monolithic Kernel with Loadable Modules

A monolithic kernel runs almost everything -- drivers, filesystem code, network stack -- in kernel space (ring 0), as one large body of code. A microkernel keeps the kernel minimal (scheduling, basic memory management, IPC) and pushes drivers/filesystems into userspace processes talking to the tiny kernel core via message passing.

Linux is architecturally a monolithic kernel -- drivers and filesystems run in ring 0 -- but gained the ability to load and unload modules at runtime without a full recompile or reboot.

[OS] -> this distinction matters for failure domains -- in a true microkernel, a buggy driver crash is (in principle) isolated to its own process, while in Linux's model a buggy kernel module runs with full ring-0 privilege and can crash the entire kernel (a kernel panic), since there is no privilege boundary between the module and the rest of the kernel once loaded.

Comparison: Monolithic vs Microkernel vs Linux's Hybrid Model

Aspect	Pure Monolithic	Microkernel	Linux (Monolithic+Modules)
Where drivers run	Kernel space (ring 0)	Userspace, isolated processes	Kernel space (ring 0)
Failure isolation	None	High	None
Performance	Fast (no IPC overhead)	Slower (message-passing overhead)	Fast
Runtime extensibility	Typically needs rebuild/reboot	N/A -- drivers are just processes	Yes -- loadable/unloadable live

15.2 Inspecting and Managing Modules

```
lsmod
```

lsmod -> list every kernel module currently loaded, size, and what depends on it

```
sudo modprobe nvidia
sudo modprobe -r nvidia
sudo insmod ./mymodule.ko
sudo rmmod mymodule
```

modprobe nvidia -> load by NAME, auto-resolving dependencies -- the apt/dnf equivalent

modprobe -r nvidia -> unload a module and, if safe, its unneeded dependencies

insmod ./mymodule.ko -> load from a SPECIFIC FILE PATH -- no dependency resolution

rmmod mymodule -> unload by name directly, no dependency awareness

Comparison: modprobe vs insmod/rmmod

Aspect	modprobe	insmod / rmmod
Dependency resolution	Yes -- auto loads/unloads dependencies	No -- fails if a dependency isn't loaded
How it finds modules	By name, standard module directories	Requires an exact file path
Analogous to (Section 10)	apt/dnf (high-level)	dpkg/rpm (low-level)

15.3 Inspecting a Module: modinfo

```
modinfo nvidia
```

modinfo -> show a module's metadata: version, author, license, description, parameters -- without loading it

⚠ NOTE: The license field modinfo reports isn't just informational -- the kernel checks it. A module without a GPL-compatible license loads in a 'tainted' state (visible in dmesg)

and /proc/sys/kernel/tainted), signaling to kernel developers that a non-GPL module is a plausible culprit in a crash.

15.4 (Optional) Writing a Minimal Module

Conceptually, a minimal 'Hello World' kernel module is a small C file with two mandatory entry points -- an init function (run on load) and an exit function (run on unload) -- compiled against the running kernel's headers using a Makefile that hooks into the kernel's own build system (kbuild), then loaded with insmod and observed via dmesg, since a kernel module writes to the kernel's own message buffer rather than a terminal.

⚠ NOTE: Writing kernel modules requires exact kernel-header version matching, and a mistake can crash the entire running kernel -- a genuinely different risk profile from every other topic in this module. Treat reading about the mechanism as sufficient at this stage rather than compiling/loading on anything but a disposable VM.

Reference Pattern: Module Management Command Parity

GENERAL: file/name only)	HIGH-LEVEL (deps resolved)	LOW-LEVEL (exact
Load <path/to/file.ko>	modprobe <name>	insmod
Unload	modprobe -r <name>	rmmod <name>
Inspect (no loading)	modinfo <name>	--
List loaded	lsmod	--

Tie-back

This connects to Section 2's kernel/userspace boundary: a loaded module executes in ring 0 with the same unrestricted privilege as code compiled into the kernel from the start. It parallels Section 10's package management, and the dmesg/tainted-kernel behavior connects to Section 12's /proc/sys/kernel tunables.

Key Takeaway

Linux's core architecture is monolithic -- drivers run in ring 0 with no privilege isolation -- but it added the ability to load/unload kernel-space code as modules at runtime without recompiling or rebooting. A kernel module runs with the exact same unrestricted privilege as the kernel itself -- there is no sandbox.

Section 15 Summary Table

Command	Purpose
lsmod	List currently loaded modules
modprobe <name>	Load a module by name, with dependency resolution
modprobe -r <name>	Unload a module (and unneeded dependencies)
insmod <path>	Load a module from an exact file path
rmmod <name>	Unload a module by name directly
modinfo <name>	Inspect a module's metadata without loading it

SECTION 16: cgroups & Namespaces [GAP-FILLED ADDITION]

16.1 Why This Section Exists

This is the most consequential gap in a module titled 'Kernel Internals.' Permissions, users, and process control govern what a process is allowed to do and as whom. cgroups and namespaces answer a different question: how much of the system's resources can a process consume, and how much of the system can it even see. Together these are the actual foundation containers (Docker, Kubernetes) are built on -- a container is just a regular Linux process with cgroups and namespaces applied to it.

16.2 Namespaces: Controlling What a Process Can See

A namespace wraps a global kernel resource so a process inside sees its own isolated instance of that resource, instead of the real, system-wide one.

Namespace	Isolates
PID	Process IDs -- a process can be PID 1 inside its namespace
NET	Network interfaces, routing tables, ports
MNT (mount)	The mount table -- a private root filesystem view
UTS	Hostname and domain name
IPC	System V IPC objects and POSIX message queues
USER	UID/GID mappings -- can be 'root' inside while mapping to unprivileged on the host

```
unshare --pid --net --mount --fork bash
```

unshare -> run a command in one or more NEW namespaces

--pid -> new PID namespace -- this shell becomes PID 1 inside it

--net -> new network namespace

--mount -> new mount namespace

--fork -> fork a new process to run the command, required with --pid

[OS] -> this is why 'ps aux' inside a Docker container shows only a handful of processes instead of the host's full list -- the container process is in its own PID namespace and genuinely cannot see processes outside it, unlike permissions which restrict what you can DO to something you CAN see.

⚠ NOTE: The USER namespace is the most misunderstood. A process can appear to be root (UID 0) inside its own user namespace while the kernel maps that UID to an unprivileged host UID -- this is precisely what lets 'rootless' containers exist.

16.3 cgroups: Controlling How Much a Process Can Consume

Control groups (cgroups) answer 'how much CPU, memory, disk I/O, and network bandwidth can this process or group actually use.' A process can see the entire system yet be strictly capped in what it consumes -- namespaces and cgroups are complementary.

```
ls /sys/fs/cgroup/  
cat /sys/fs/cgroup/memory.max  
echo "500M" > /sys/fs/cgroup/mygroup/memory.max  
/sys/fs/cgroup/ -> cgroups exposed as a virtual filesystem -- continuation of Section 12's /sys pattern
```

memory.max -> memory ceiling for processes in this cgroup -- kernel OOM-kills a process exceeding it

[PERF] -> this is the concrete kernel mechanism behind Docker's --memory and --cpus flags - a direct extension of the scheduler/memory-management concepts from Section 2, with the kernel itself doing enforcement rather than a userspace watchdog polling and killing things after the fact.

Comparison: Namespaces vs cgroups

Aspect	Namespaces	cgroups
Question answered	What can this process SEE?	How much can this process USE?
Isolates	PIDs, network, mounts, hostname, IPC, UIDs	CPU, memory, disk I/O, network bandwidth
Enforcement	Isolated views of kernel data structures	Hard resource limits enforced by scheduler/mem manager
Exposed as	/proc/<pid>/ns/*	/sys/fs/cgroup/
What breaks without it	A process could see/interfere with the whole host	One process could starve every other process

16.4 How This Adds Up to 'A Container'

A container is not a distinct kernel object type -- there is no container() syscall. A container runtime, on the host kernel: (1) creates a set of new namespaces (PID, NET, MNT, UTS, IPC, USER) for the process; (2) places that process into a cgroup with defined CPU/memory limits; (3) sets up a root filesystem for the MNT namespace (typically a layered image); (4) executes the target process (exec) inside all of the above. Every container running on a machine is, from the host kernel's own perspective, just an ordinary process -- visible in the host's own ps aux with an ordinary PID, subject to the same signals and syscalls as anything else.

Reference Pattern: Namespace Types and Their /proc Identifiers

```
GENERAL:      /proc/<pid>/ns/<namespace_type>  
  
EXAMPLE 1:    /proc/1234/ns/pid      -> which PID namespace process 1234  
belongs to  
EXAMPLE 2:    /proc/1234/ns/net      -> which network namespace process 1234  
belongs to  
EXAMPLE 3:    /proc/1234/ns/mnt      -> which mount namespace process 1234  
belongs to
```

```
EXAMPLE 4: /proc/1234/ns/user -> which user namespace process 1234
belongs to
```

Tie-back

This is the direct capstone of the kernel-internals theme: `/sys/fs/cgroup` is the same virtual-filesystem pattern from Section 12, applied to resource limits; namespaces reframe Section 14's process tree (PID namespaces) and Section 5's mount table (MNT namespaces); USER namespaces extend Section 9's UID/GID model with an additional mapping layer.

Key Takeaway

Docker doesn't use a container-specific kernel feature -- it combines namespaces (isolating what a process can see) with cgroups (limiting what a process can consume), then runs an ordinary process inside that isolated, resource-limited environment. A container is not a kernel object -- it is an ordinary process wrapped in namespaces and a cgroup, both regular kernel features you can inspect directly without Docker at all.

Section 16 Summary Table

Concept	Kernel Feature	Answers
Isolation of view	Namespaces (PID, NET, MNT, UTS, IPC, USER)	What can this process see?
Resource limiting	cgroups	How much CPU/memory/I-O can this process use?
Exposed as	<code>/proc/<pid>/ns/*</code> , <code>/sys/fs/cgroup/</code>	Both continue the <code>/proc//sys</code> virtual filesystem pattern
Real-world application	Containers (Docker, Kubernetes, containerd)	Process + namespaces + cgroup = 'a container'

SECTION 17: Module 1 Capstone Checkpoint

A hands-on checkpoint, not new theory -- every task pulls together tools and concepts from Sections 1-16. Do these on a real Linux VM (a disposable one is ideal, especially for the cgroup/module tasks).

17.1 Task: Narrate the Full Boot Process

```
journalctl -b
```

Identify, in your own words, where each Section 3 stage appears in the log: firmware handoff, kernel decompression, `initramfs` mounting the real root, and `systemd` taking over and starting units in sequence.

17.2 Task: Trace ≥ 5 Syscalls of a Simple Command

```
strace ls
```

Identify at least 5 distinct syscalls and explain each -- expect `execve`, `openat`, `mmap`, `read`, `close`, `write`. Exercises Section 13's syscall categories and return-value convention.

17.3 Task: Write and Deploy a Custom .service Unit

```
sudo systemctl daemon-reload
sudo systemctl enable --now myscript.service
sudo systemctl status myscript.service
```

daemon-reload -> tells systemd to re-read unit files from disk after creating/editing one

Requirements: starts on boot (enable), restarts automatically on failure (Restart=on-failure). Kill the process manually (kill -9) and confirm systemd restarts it.

17.4 Task: Explore /proc/<your-shell-pid>/

```
echo $$
ls /proc/$$/
```

Explain 5 files/directories inside (status, fd/, maps, cmdline, environ, cwd, exe). Also check /proc/\$\$/ns/ and note the namespace identifiers -- on a normal login shell these match the host's default namespaces.

17.5 Task: Explore /sys/fs/cgroup

```
ls /sys/fs/cgroup/
cat /sys/fs/cgroup/cpu.max 2>/dev/null || echo "check your cgroup version (v1 vs v2 paths differ)"
```

Identify which cgroup your shell's PID belongs to (cat /proc/\$\$/cgroup), and explain what memory.max and cpu.max would do if set to a finite value.

17.6 Task: One-Paragraph Summary

Write, in your own words: 'How does a Linux system go from powered-off to a running shell prompt, and where does the kernel take over from firmware?' -- the best test of whether Sections 2, 3, and 13 have connected into one coherent mental model.

Full Module Capstone Checklist

#	Task	Sections Exercised
1	Narrate full boot sequence via journalctl -b	3
2	Trace >=5 syscalls of a simple command with strace	13
3	Write+deploy a custom .service (boots, restarts on failure)	11, 14
4	Explore and explain 5 files inside /proc/<pid>/	12
5	Explore /sys/fs/cgroup and identify your shell's cgroup	16
6	One-paragraph boot-to-shell-prompt summary	2, 3, 13